

CASE STUDY :

SaigonRide - Distributed Vehicle Rental System

Course: Introduction to Software Engineering | **Semester:** 2 (2025-2026)

1. Context & Background

Ho Chi Minh City is rapidly modernizing its public transit ecosystem to reduce traffic congestion and emissions. A key part of this initiative is **SaigonRide**, a distributed network of public bicycles and electric scooters. Unlike traditional rentals, users can unlock a vehicle at Station A (e.g., Ben Thanh) and return it to Station B (e.g., Thao Dien).

However, the current digital infrastructure is fragmented. The system fails to bridge the gap between two primary user groups: **Local Commuters** (who rely on local e-wallets like MoMo and VNPAY) and **Foreign Tourists** (who require international gateways like Apple Pay and PayPal, plus passport verification). Furthermore, the city struggles with fleet balancing, often ending up with completely empty or overflowing stations.

2. Development Challenges

As second-year software engineering students, tackling this system presents several core technical and design challenges:

- **Real-Time State Management:** Tracking the physical state of tangible entities (Vehicles) as they shift rapidly between "Available," "In-Transit," and "Maintenance."
- **Algorithmic Pricing Logic:** Calculating fares dynamically based on time (minutes used), vehicle category (E-scooter vs. Standard Bike), and location-based inventory levels.
- **Architectural Separation of Concerns:** Designing a multi-gateway payment system without creating "spaghetti code." We must use the Strategy Design Pattern so local and international payments are handled cleanly.
- **Actor Generalization:** Handling the differing authentication and onboarding flows for local citizens versus foreign tourists within the same database schema.

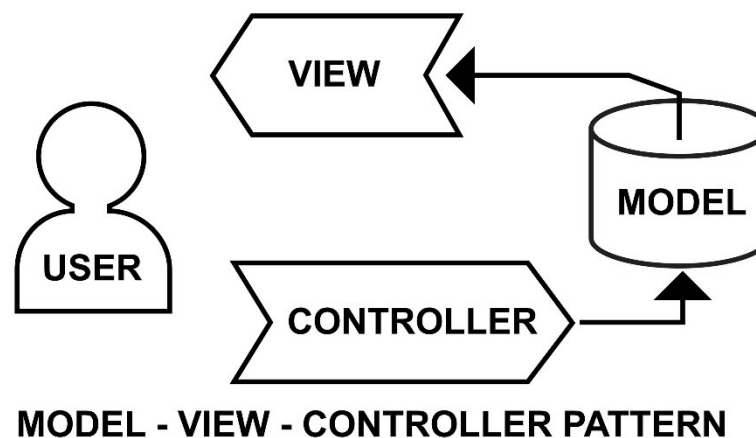
3. Project Vision & Objectives

Vision: To engineer a unified, scalable, and user-friendly smart mobility platform that seamlessly integrates fleet inventory management, dynamic pricing, and localized/international payment processing.

Key Objectives:

1. **Full CRUD Implementation:** Ensure complete Create, Read, Update, and Delete functionality for core entities (Users, Vehicles, Stations, Rentals).
2. **Incentivized Fleet Rebalancing:** Successfully implement a Loyalty Point Engine that applies to a 15% discount to users who return vehicles to low-inventory stations.
3. **Robust Quality Assurance:** Achieve a highly tested codebase by applying Equivalence Partitioning (EP) and Boundary Value Analysis (BVA) to all pricing algorithms.
4. **Actionable Reporting:** Provide System Admins with accurate, real-time data through at least two distinct operational reports (e.g., Revenue by Category, Station Utilization).

4. Overall Project Requirements



A. Functional Requirements (FRs)

- **FR1:** The system shall calculate fares at 500 VND/min for Standard Bikes and 1,500 VND/min for E-Scooters.
- **FR2:** The system shall apply a 15% discount to the total fare if a vehicle is returned to a station currently at < 20% capacity.
- **FR3:** The system shall allow Local Commuters to check out using MoMo, VNPAY, or Cash.
- **FR4:** The system shall allow Foreign Tourists to checkout using Apple Pay, PayPal, or Cash.

- **FR5:** The system shall generate at least two administrative reports (Revenue and Inventory).

B. Non-Functional Requirements (Quality Attributes)

- **Usability:** The UI must be responsive (using Bootstrap or any Front-end tools) and support bilingual toggling (English/Vietnamese).
- **Performance:** Fare calculation and discount processing must execute in under 1.5 seconds.
- **Maintainability:** The codebase must strictly adhere to a 3-Tier or MVC architecture.

5. Scoring Criteria (The "Tiered" Target)

Our team must self-select a technological tier, which caps our maximum possible grade based on technical complexity. We are evaluated on a 100% scale across 4 phases:

- **Tier 1 (Max 7.0):** C# WinForms (3-Tier)
- **Tier 1B (Max 7.5):** Core PHP / MySQL (MVC)
- **Tier 2 (Max 8.5):** ASP.NET Web Forms/MVC Database-First
- **Tier 3 (Max 9.5):** ASP.NET MVC Code-First + Entity Framework
- **Tier 4 (Max 10.0):** Tier 3 + Live API Integration (VNPay/PayPal) + Automated Testing.

Weight Distribution:

- **Part 1 (20%):** Requirements Doc (15%) + Solution Pitch (5%)
- **Part 2 (25%):** Design Doc & Figma (20%) + Design Review (5%)
- **Part 3 (35%):** Code, GitHub, QA Testing (30%) + Live Demo (5%)
- **Final Submission (20%):** Consolidated Report with Traceability/Lessons Learnt (10%) + Video Demo (10%)

6. Necessary Actions & SDLC Roadmap



To successfully deliver this software, our team will execute the following actions across a 7-week Agile timeline:

- **Weeks 1-2 (Analysis):** * Assign one main Use Case (with full CRUD) to each team member.
 - Draft the Proposal, FRs, and NFRs.
- **Weeks 3-4 (Design):** * Create CRC Cards for object planning.
 - Draw UML Use Case, Sequence, and Class Diagrams.
 - Design a 4-stage interactive Figma prototype (Sketch -> Prototype) utilizing Bootstrap grid layouts.
- **Weeks 5-7 (Implementation & QA):** * Code the application using strict MVC/3-Tier separation.
 - Commit code regularly to GitHub (>10 commits per student).
 - Write a Test Plan and execute Unit Tests on dynamic pricing logic.
 - Create a “Requirements Traceability Matrix (RTM)” to prove our code meets the initial requirements.
 - Compile the final PDF report, record the video demo, and present the live working product.

(Please refer to the separate "Project Requirements" document for detailed grading rubrics, submission guidelines, and specific deliverables for each tier).